As speed is one of the unique selling points (USP) of H2O, we decided to test its performance.

Considering that we had a customer segmentation solution implementation using the KMeans algorithm provided by Apache Mahout, we wanted to compare its performance with the KMeans implementation provided by H2O. While H2O documentation provides information about the KMeans algorithm, I was not able to locate suitable examples that would allow me to execute it on a test data set. I was able to execute the algorithm on the test data using the H2O user interface (known as Flow), but this method was obviously not scalable as it cannot be integrated easily with enterprise applications. For this purpose, I wanted to invoke the algorithm from a Java application, enabling easier integration with enterprise applications.

Using the KMeans algorithm

I searched for samples that would show me Let's Try use the library, but with little success. Finally, a colleague found an example written in Scala, which was part of the Sparkling Water library (a new library being developed by combining Apache Spark and H2O libraries). I have mapped the Scala source code to Java and have added a print statement that displays the result of the clustering (this part was missing from the sample). The sample application is shown in the following sections.

The main method

In the main method of the sample (as depicted in Snippet 1), we need to set up the H2O instance, which in this case will be on the local machine (standalone mode of H2O, one node cluster).

```
// Setup cloud name
String[] args = new String[]
// Build a cloud of 1
H2O.waitForCloudSize(1, 10*1000);
```

Snippet 1: The main method

Parsing the data

After creating the cluster, we need to read the data. The data is an input file, in the CSV format, containing numerical information. One line of the input represents one record. Each record represents information for one customer. Thus, the input file represents the customers we wish to group into clusters using the KMeans algorithm. As depicted in Snippet 2, the file is read into a vector of values and parsed into the Frame object.

```
NFSFileVec nfs = NFSFileVec.make(f);
Frame frame = water.parser.ParseDataset.parse(Key.
make(),nfs._key);
```

Snippet 2: Parsing the data

Initialising the KMeans algorithm

After parsing the data, we set up the KMeans algorithm. In this case, we created five clusters and the number of iterations for algorithm execution was 1000. This is depicted in Snippet 3.

```
KMeansParameters params = new KMeansParameters();
params._k = 5;
params._max_iterations = 1000;
```

Snippet 3: Configuring the algorithm

Training the algorithm and executing it

Now, we have to create the KMeans model using the specified parameters. We then execute the algorithm by running the *score* method of the KMeansModel object. The *score* method executes the KMeans algorithm on the specified data (stored in the *frame* vector). The output of the algorithm is stored in another Frame object, named *output*, as depicted in Snippet 4. It is important to note that the Frame objects are stored in the memory by H2O.

```
// create the model
KMeans job = new KMeans(params);
// train the model
KMeansModel kmm = job.trainModel().get();
// execute the model on the loaded data
Frame output = kmm.score(frame);
```

Snippet 4: Creating the model and scoring it

Displaying the results

After execution, the results are available in the *output* variable. In order to display which customer record belongs to which cluster, we need to access the *vecs* variable from the *output* variable. In this sample, we are printing the result on the screen. The vector *vecs* contains as many records as were present in the input file. Each entry mentions the ID of the cluster into which the customer record has been placed, as depicted in Snippet 5.

```
Vec[] vecs = output.vecs();
for ( Vec v : vecs ) {
System.out.println(v + ", " + v.length());
for ( int i = 0; i < v.length(); i++ ) {
System.out.println(i + ": " + v.at(i));
}
}
System.out.println();
```

Snippet 5: Displaying the cluster ID for each record

The complete sample

The complete sample is presented below.